

Week 3 Lab Report: Control Flow & Functions

Course: Introduction to Big Data Analytics (AUCA)	Student Name: Kwizera Pacifique
Student ID: 28054	Branch: week3/28054-kwizera-pacifique
Notebook: Week3_Kwizera_Pacifique.ipynb	Submission: Parts 1–6 Completed (+3 Bonus)

1. Executive Summary

This report evaluates student performance metrics from `week3_students.csv` using custom Python control flow logic, defensive error-handling routines, and modular functional programming. Out of **40 raw student records**, the pipeline identified and validated **37 clean records** while gracefully surviving **3 broken records (7.5% corruption rate)** without raising unhandled exceptions or terminating early. All lab exercises, bonus requirements, and reflection tasks were completed with complete fidelity.

2. Identified Broken Records

During string-to-integer conversion of the score column (index 7), three records triggered runtime conversion errors:

Student ID	Student Name	Program	District	Att %	Score Entry	Underlying Fault
AUCA008	Emmanuel Nsengiyumva	Accounting	Rubavu	65%	"N/A"	Non-numeric placeholder string
AUCA020	Samuel Rukundo	Accounting	Kicukiro	94%	""	Missing value (empty string)
AUCA032	Gilbert Uwituze	Accounting	Rubavu	63%	"absent"	Textual attendance string in numeric field

3. How the Code Survived Dirty Data

Standard invocation of `int(score_str)` triggers an immediate `ValueError` upon encountering empty strings or non-numeric characters. To provide robust fault tolerance, the ingestion loop encapsulated type conversion within an explicit `try...except ValueError` block:

```
# Defensive Ingestion Pipeline (Exercise 5.1)
try:
    score = int(score_str)
    total_valid_score += score
    valid_count += 1
except ValueError:
    bad_count += 1
    bad_ids.append(student_id)
```

- Exception Interception:** The `try-except` construct safely caught runtime conversion exceptions without terminating the script.
- Mathematical Integrity:** Broken entries were barred from score accumulators, preventing skewing of class and district averages.
- Traceability:** Corrupted student IDs were isolated in `bad_ids` for auditing.

4. Key Analytics Results

Metric	Result	Context
Total Processed Rows	40	Full cohort from <code>week3_students.csv</code>
Valid Clean Records	37 (92.5%)	Successfully parsed scores
Corrupted Records	3 (7.5%)	Safely bypassed via exception handling
Cohort Average Score	75.3	Sum of 2,786 points / 37 valid records
Pass Rate (≥ 50)	94.6% (35 students)	Only 2 valid students failed (< 50)
Top Student	Bonaventure Nkurunziza	Score: 95 (Grade: A , ID: AUCA024)
Top Performing District	Muhanga	Highest district average: 87.00

District Performance Breakdown:

District	Average Score	District	Average Score
Muhanga	87.00 (🏆 Highest)	Nyarugenge	74.80
Nyagatare	82.80	Huye	68.40
Gasabo	80.40	Musanze	65.20
Rubavu	78.67	Kicukiro	64.00

5. Reflection (Part 6)

1. Data Trust Threshold & Remediation

"Your pipeline skipped 3 broken records out of 40 (7.5%). At what percentage would you stop trusting the dataset — and what would you do instead of skipping?"

Analysis: At 7.5%, dropping problematic records is reasonable for preliminary analysis. However, if corrupted records reached **15% or higher**, automated skipping would introduce severe *selection bias*. At that threshold, rather than skipping:

- **Audit Upstream Pipelines:** Query source registration databases to pinpoint input validation regressions or CSV delimiter issues.
- **Statistical Imputation:** Apply domain-informed imputation (substituting cohort medians or district averages) marked with an `is_imputed` boolean flag.
- **Dead-Letter Logging:** Direct defective rows into a quarantine queue for administrative reconciliation.

2. Session Learning & Surprises

"Describe one thing from today's session that surprised or confused you."

Analysis: It was eye-opening how fragile basic operations like `int()` can be in production environments—a single value like `"absent"` halts an entire system if unshielded. Implementing defensive programming with `try...except` demonstrated how a few targeted lines of error handling transform fragile scripts into resilient production-ready pipelines.

6. Submission Checklist & Screenshots Directory

- ✓ **Notebook Renamed:** `Week3_Kwizera_Pacifique.ipynb` in branch root
- ✓ **Flawless Execution:** All parts (1 to 5 + Bonus) execute cleanly without errors
- ✓ **Broken Records Identified:** AUCA008, AUCA020, AUCA032 catalogued
- ✓ **Part 6 Answered:** Full analytical reflections included
- ✓ **9 Uncropped Screenshots:** Saved in `screenshots/` folder
- ✓ **Pull Request Ready:** Branch `week3/28054-kwizera-pacifique` ready for PR